

UNIVERSIDAD
NACIONAL
DE COLOMBIA

ACTIVIDAD 3 DE PROGRAMACION ORIENTADA A OBJETOS

Presentado por: Maria Paula Barrera Avella – 1001220174

Presentado a: Walter Hugo Arboleda Mazo

Asignatura: Programación orientada a objetos

Medellin, Antioquia

Fecha: 22 de octubre

2026

Ejercicio 8.2

```
# EJERCICIO 8.2

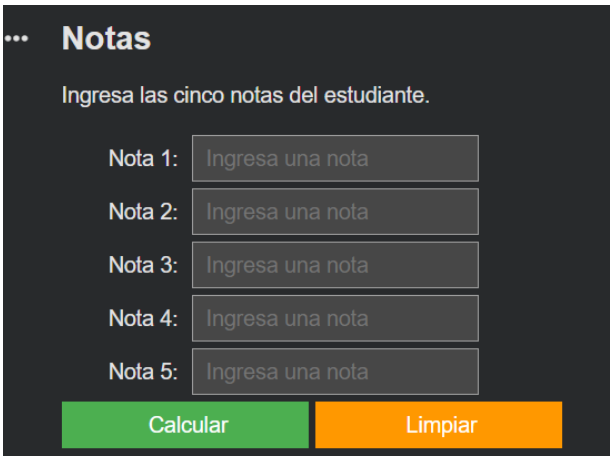
import math
import ipywidgets as widgets
from IPython.display import display, clear_output
# Clase Notas
class Notas:
    def __init__(self, lista_notas):
        self.lista_notas = lista_notas
    def calcular_promedio(self):
        suma = 0
        for nota in self.lista_notas:
            suma += nota
        return suma / len(self.lista_notas)
    def calcular_desviacion(self):
        promedio = self.calcular_promedio()
        suma = 0
        for nota in self.lista_notas:
            suma += (nota - promedio) ** 2
        return math.sqrt(suma / len(self.lista_notas))
    def calcular_menor(self):
        menor = self.lista_notas[0]
        for nota in self.lista_notas:
            if nota < menor:
                menor = nota
        return menor
    def calcular_mayor(self):
        mayor = self.lista_notas[0]
        for nota in self.lista_notas:
            if nota > mayor:
                mayor = nota
        return mayor
# Clase VentanaPrincipal
class VentanaPrincipal:
    def __init__(self):
        self.campos = []
        # Crear los cinco campos para ingresar las notas
        for i in range(5):
            campo = widgets.Text(
                description=f"Nota {i + 1}:",
                placeholder="Ingresa una nota",
                style={"description_width": "70px"},
                layout=widgets.Layout(width="250px")
            )
            self.campos.append(campo)
        # Boton para calcular
        self.boton_calcular = widgets.Button(
            description="Calcular",
            button_style="success",
            icon="calculator"
```

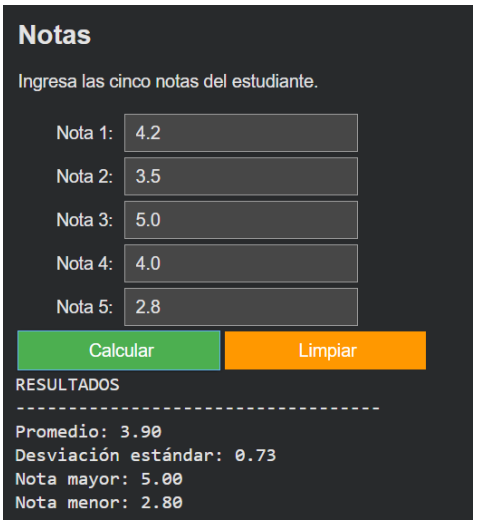
```
        )
        # Boton para limpiar
        self.boton_limpiar = widgets.Button(
            description="Limpiar",
            button_style="warning",
            icon="trash"
        )
        # Area de resultados y alertas
        self.resultados = widgets.Output()
        # Asociar los botones con sus funciones
        self.boton_calcular.on_click(
            self.action_performed
        )
        self.boton_limpiar.on_click(
            self.limpiar
        )
    # Validar las notas y calcular los resultados
    def action_performed(self, boton):
        with self.resultados:
            clear_output(wait=True)
            try:
                lista_notas = []
                # Validar las cinco notas
                for i, campo in enumerate(
                    self.campos, start=1
                ):
                    texto = (
                        campo.value.strip().replace(",", ".")
                    )
                    # Validar que la nota no este vacia
                    if texto == "":
                        raise ValueError(
                            f"Alerta: no has ingresado "
                            f"la nota {i}."
                        )
                    # Validar que el dato sea numerico
                    try:
                        nota = float(texto)
                    except ValueError:
                        raise ValueError(
                            f"Alerta: la nota {i} "
                            f"debe ser un numero."
                        )
                    # Validar que el numero sea finito
                    if not math.isfinite(nota):
                        raise ValueError(
                            f"Alerta: la nota {i} "
                            f"no es un numero valido."
                        )
                    # Validar el rango permitido
                    if nota < 0 or nota > 5:
```

```
                raise ValueError(
                    f"Alerta: la nota {i} "
                    f"debe estar entre 0 y 5."
                )
            lista_notas.append(nota)
        # Crear el objeto Notas
        notas = Notas(lista_notas)
        # Mostrar los resultados
        print("RESULTADOS")
        print("-" * 35)
        print(
            f"Promedio: "
            f"{notas.calcular_promedio():.2f}"
        )
        print(
            f"Desviacion estandar: "
            f"{notas.calcular_desviacion():.2f}"
        )
        print(
            f"Nota mayor: "
            f"{notas.calcular_mayor():.2f}"
        )
        print(
            f"Nota menor: "
            f"{notas.calcular_menor():.2f}"
        )
    except ValueError as error:
        print(f"\n{error}")
# Limpiar los campos y los resultados
def limpiar(self, boton):
    for campo in self.campos:
        campo.value = ""
    with self.resultados:
        clear_output(wait=True)
# Mostrar la interfaz
def mostrar(self):
    titulo = widgets.HTML(
        "<h2>Notas</h2>"
        "<p>Ingresa las cinco notas del estudiante.</p>"
        "<p>Las notas deben estar entre 0 y 5.</p>"
    )
    display(titulo)
    display(widgets.VBox(self.campos))
    display(
        widgets.HBox([
            self.boton_calcular,
            self.boton_limpiar
        ])
    )
    display(self.resultados)
```

```
# Clase Principal
class Principal:
    @staticmethod
    def main():
        ventana = VentanaPrincipal()
        ventana.mostrar()
# Ejecutar el programa
Principal.main()
```

Interfaz de usuario





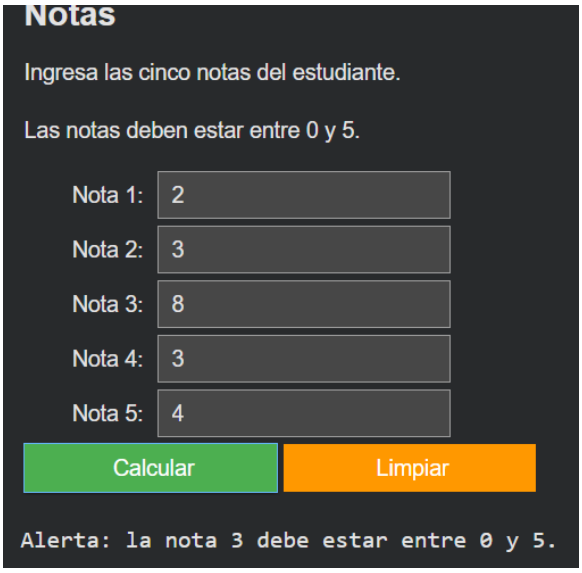


Diagrama de clases

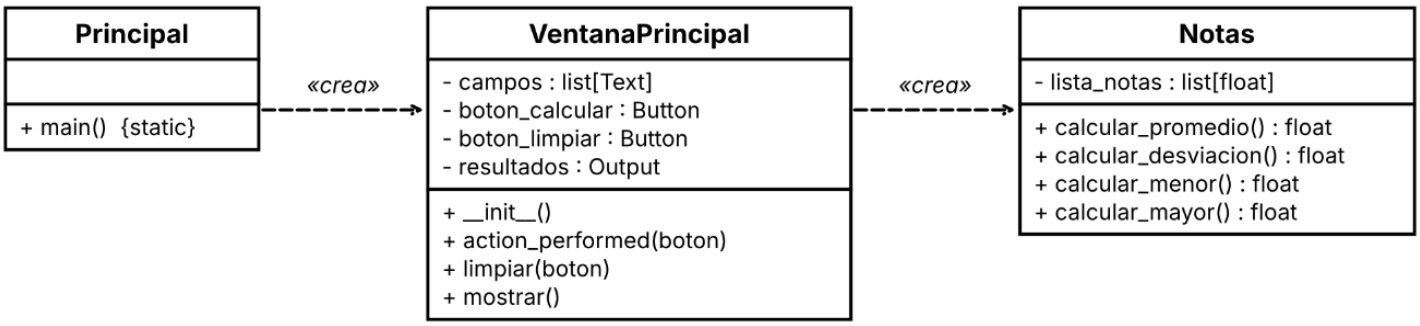
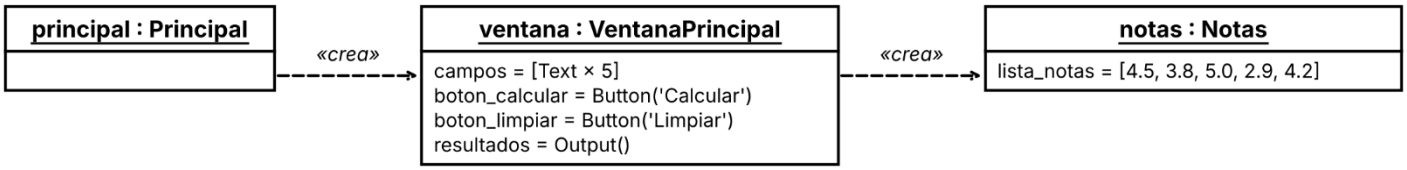
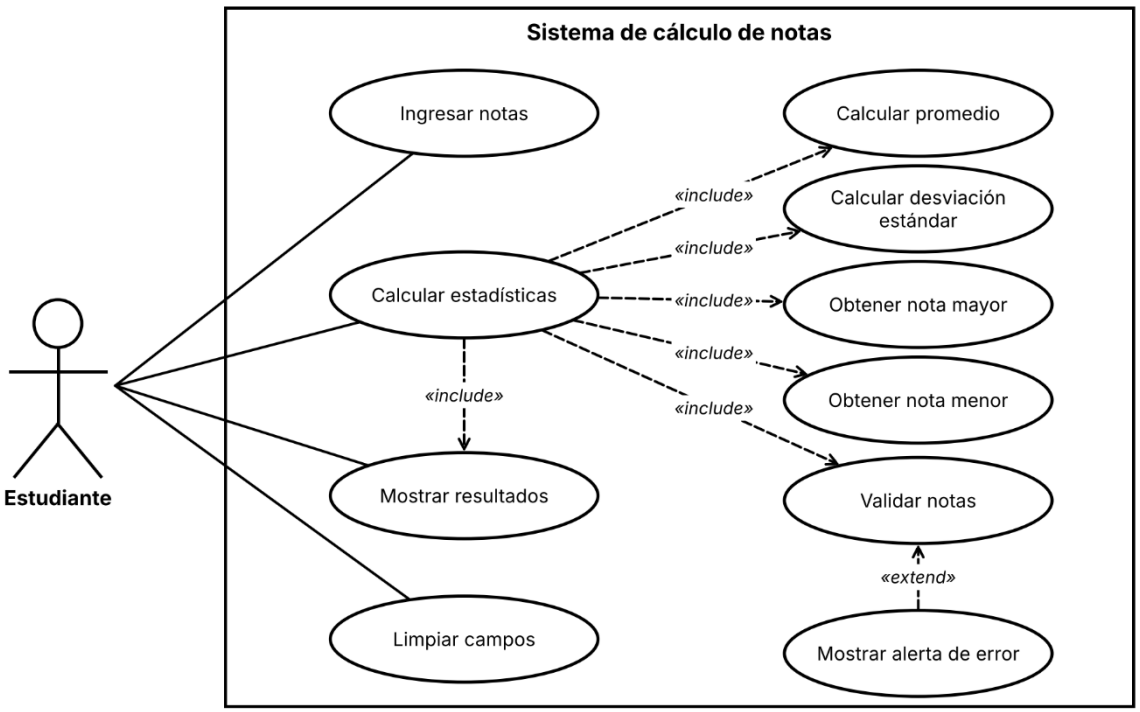


Diagrama de objetos



Casos de uso



Ejercicio 8.3

```
# EJERCICIO 8.3
import math
import os
import ipywidgets as widgets
import matplotlib.pyplot as plt
from abc import ABC, abstractmethod
from IPython.display import display, clear_output
from PIL import Image
import pillow_avif
# CLASE BASE: FIGURAGEOMETRICA
class FiguraGeometrica(ABC):
    def __init__(self):
        self._volumen = 0.0
        self._superficie = 0.0
    def set_volumen(self, volumen):
        self._volumen = volumen
    def set_superficie(self, superficie):
        self._superficie = superficie
    def get_volumen(self):
        return self._volumen
    def get_superficie(self):
        return self._superficie
    @abstractmethod
    def calcular_volumen(self):
        pass
    @abstractmethod
    def calcular_superficie(self):
        pass
# CLASE CILINDRO
class Cilindro(FiguraGeometrica):
    def __init__(self, radio, altura):
        super().__init__()
        self.radio = self.validar_positivo(radio, "radio")
        self.altura = self.validar_positivo(altura, "altura")
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())
    @staticmethod
    def validar_positivo(valor, nombre):
        valor = float(valor)
        if not math.isfinite(valor) or valor <= 0:
            raise ValueError(
                f"El valor de {nombre} debe ser mayor que cero."
            )
        return valor
    def calcular_volumen(self):
        return math.pi * self.radio**2 * self.altura
    def calcular_superficie(self):
        return (
```

```
            2 * math.pi * self.radio * self.altura
            + 2 * math.pi * self.radio**2
        )
# CLASE ESFERA
class Esfera(FiguraGeometrica):
    def __init__(self, radio):
        super().__init__()
        self.radio = Cilindro.validar_positivo(radio, "radio")
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())
    def calcular_volumen(self):
        return (4 / 3) * math.pi * self.radio**3
    def calcular_superficie(self):
        return 4 * math.pi * self.radio**2
# CLASE PIRAMIDE DE BASE CUADRADA
class Piramide(FiguraGeometrica):
    def __init__(self, base, altura, apotema):
        super().__init__()
        self.base = Cilindro.validar_positivo(base, "base")
        self.altura = Cilindro.validar_positivo(altura, "altura")
        self.apotema = Cilindro.validar_positivo(
            apotema, "apotema"
        )
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())
    def calcular_volumen(self):
        return self.base**2 * self.altura / 3
    def calcular_superficie(self):
        return self.base**2 + 2 * self.base * self.apotema
# CLASE CUBO
class Cubo(FiguraGeometrica):
    def __init__(self, lado):
        super().__init__()
        self.lado = Cilindro.validar_positivo(lado, "lado")
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())
    def calcular_volumen(self):
        return self.lado**3
    def calcular_superficie(self):
        return 6 * self.lado**2
# CLASE PRISMA RECTANGULAR
class Prisma(FiguraGeometrica):
    def __init__(self, largo, ancho, altura):
        super().__init__()
        self.largo = Cilindro.validar_positivo(largo, "largo")
        self.ancho = Cilindro.validar_positivo(ancho, "ancho")
        self.altura = Cilindro.validar_positivo(altura, "altura")
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())
    def calcular_volumen(self):
        return self.largo * self.ancho * self.altura
```

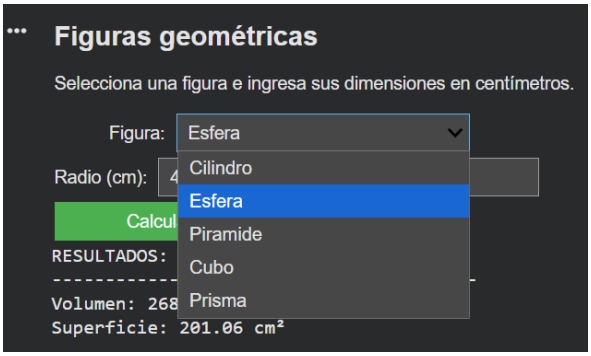
```
    def calcular_superficie(self):
        return 2 * (
            self.largo * self.ancho
            + self.largo * self.altura
            + self.ancho * self.altura
        )
# FUNCION PARA MOSTRAR LAS IMAGENES REALES
def mostrar_imagen(tipo):
    archivos = {
        "Esfera": "/content/Esfera.jpg",
        "Cilindro": "/content/cilindro.avif",
        "Piramide": "/content/piramide.jpg",
        "Cubo": "/content/cubo.jpg",
        "Prisma": "/content/Prisma.jpg"
    }
    ruta = archivos[tipo]
    if not os.path.isfile(ruta):
        print(f"No se encontro la imagen: {ruta}")
        return
    imagen = Image.open(ruta)
    imagen.thumbnail((350, 350))
    display(imagen.copy())
# CLASE VENTANAFIGURA
class VentanaFigura:
    def __init__(self, tipo, campos):
        self.tipo = tipo
        self.campos = campos
    def obtener_valores(self):
        valores = {}
        for nombre, campo in self.campos.items():
            texto = campo.value.strip().replace(",", ".")
            if not texto:
                raise ValueError(
                    f"Debes ingresar el valor de {nombre}."
                )
            try:
                valor = float(texto)
            except ValueError:
                raise ValueError(
                    f"El valor de {nombre} debe ser numerico."
                )
            if not math.isfinite(valor):
                raise ValueError(
                    f"El valor de {nombre} no es valido."
                )
            valores[nombre] = valor
        return valores
# CLASE VENTANAPRINCIPAL
class VentanaPrincipal:
    def __init__(self):
        self.tipo = widgets.Dropdown(
```

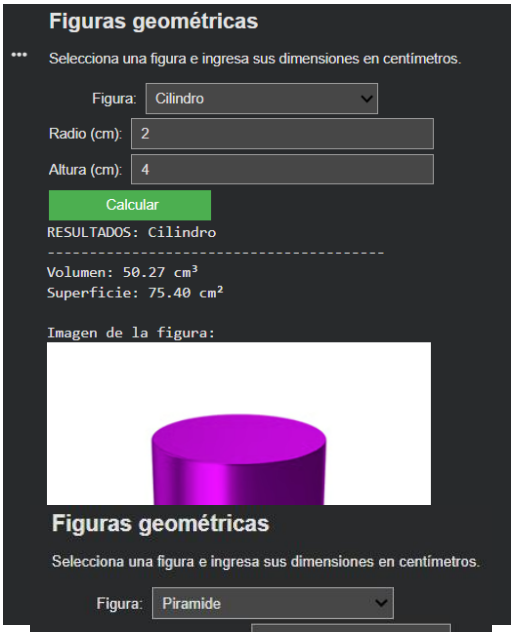
```
            options=[
                "Cilindro",
                "Esfera",
                "Piramide",
                "Cubo",
                "Prisma"
            ],
            value="Cilindro",
            description="Figura:"
        )
        self.formulario = widgets.VBox()
        self.campos = {}
        self.boton_calcular = widgets.Button(
            description="Calcular",
            button_style="success",
            icon="calculator"
        )
        self.resultados = widgets.Output()
        self.tipo.observe(
            self.actualizar_campos,
            names="value"
        )
        self.boton_calcular.on_click(
            self.action_performed
        )
        self.actualizar_campos()
    def actualizar_campos(self, cambio=None):
        definiciones = {
            "Cilindro": [
                ("radio", "Radio (cm):"),
                ("altura", "Altura (cm):")
            ],
            "Esfera": [
                ("radio", "Radio (cm):")
            ],
            "Piramide": [
                ("base", "Base cuadrada (lado en cm):"),
                ("altura", "Altura (cm):"),
                ("apotema", "Apotema (cm):")
            ],
            "Cubo": [
                ("lado", "Lado (cm):")
            ],
            "Prisma": [
                ("largo", "Largo (cm):"),
                ("ancho", "Ancho (cm):"),
                ("altura", "Altura (cm):")
            ]
        }
        self.campos = {}
```

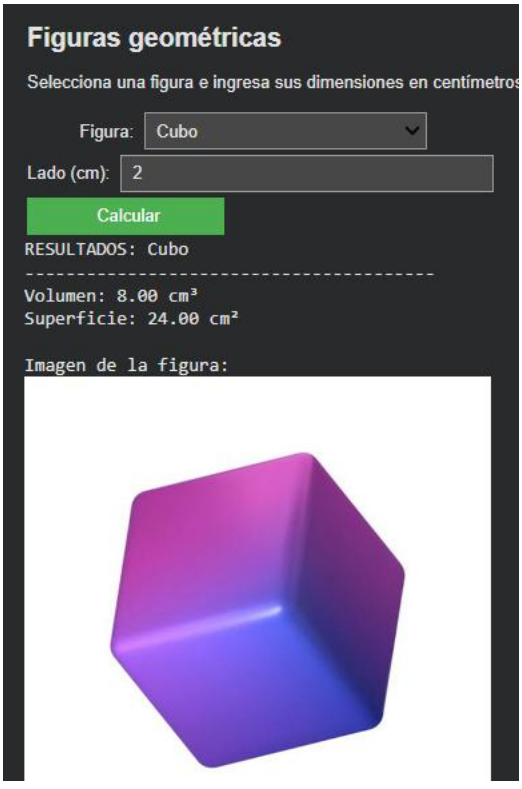
```
        for nombre, etiqueta in definiciones[self.tipo.value]:
            self.campos[nombre] = widgets.Text(
                description=etiqueta,
                placeholder="Ingresa un valor",
                style={"description_width": "initial"},
                layout=widgets.Layout(width="350px")
            )
        self.formulario.children = tuple(
            self.campos.values()
        )
        with self.resultados:
            clear_output(wait=True)
    def action_performed(self, boton):
        with self.resultados:
            clear_output(wait=True)
            try:
                ventana = VentanaFigura(
                    self.tipo.value,
                    self.campos
                )
                valores = ventana.obtener_valores()
                clases = {
                    "Cilindro": Cilindro,
                    "Esfera": Esfera,
                    "Piramide": Piramide,
                    "Cubo": Cubo,
                    "Prisma": Prisma
                }
                figura = clases[self.tipo.value](**valores)
                print(f"RESULTADOS: {self.tipo.value}")
                print("-" * 40)
                print(
                    f"Volumen: {figura.get_volumen():.2f} cm³"
                )
                print(
                    f"Superficie: "
                    f"{figura.get_superficie():.2f} cm²"
                )
                print("\nImagen de la figura:")
                # Mostrar la imagen dentro del area de resultados
                mostrar_imagen(self.tipo.value)
            except (ValueError, TypeError, OSError) as error:
                print(f"Error: {error}")
    def mostrar(self):
        display(widgets.HTML(
            "<h2>Figuras geométricas</h2>"
            "<p>Selecciona una figura e ingresa sus "
            "dimensiones en centímetros.</p>"
        ))
        display(self.tipo)
        display(self.formulario)
```

```
        display(self.boton_calcular)
        display(self.resultados)
# CLASE PRINCIPAL
class Principal:
    @staticmethod
    def main():
        ventana = VentanaPrincipal()
        ventana.mostrar()
# EJECUTAR EL PROGRAMA
Principal.main()
```

Interfaz de usuario







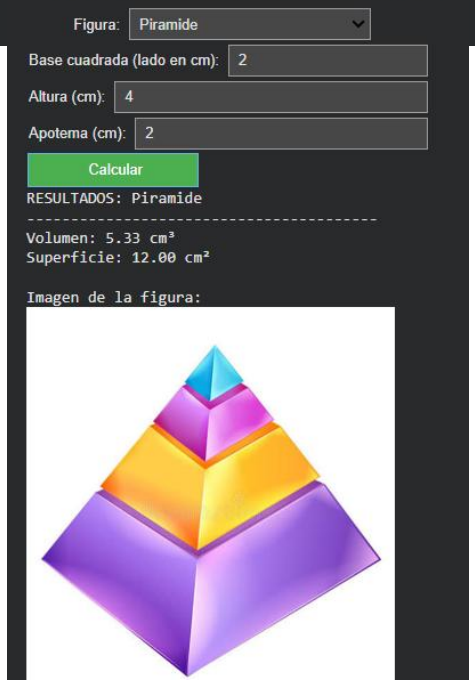
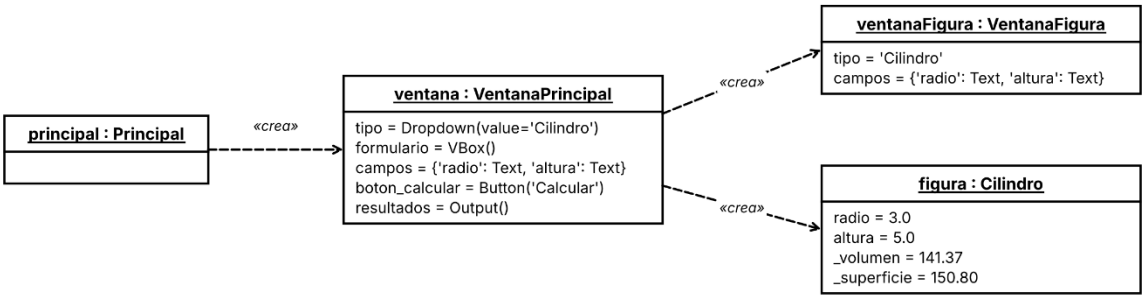
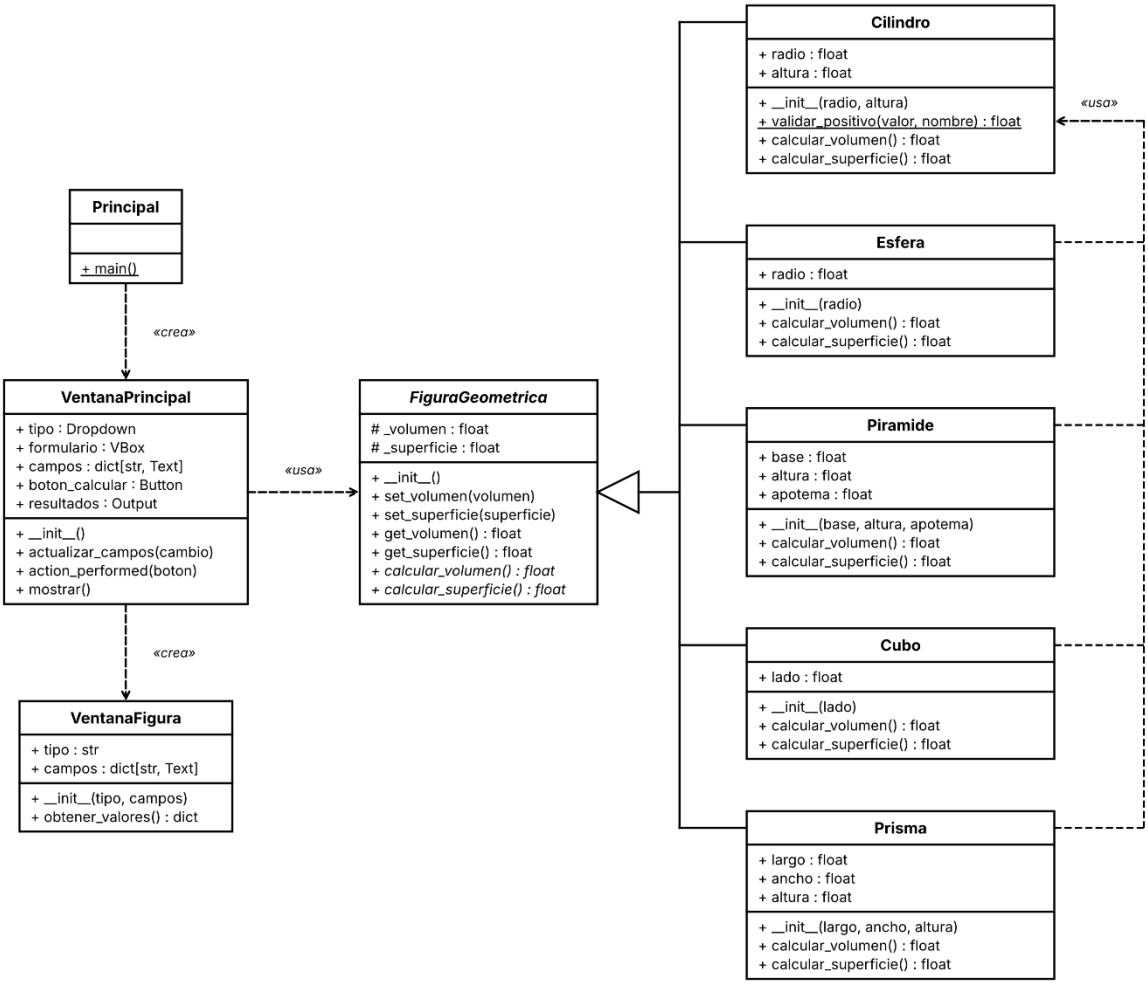


Diagrama de objetos

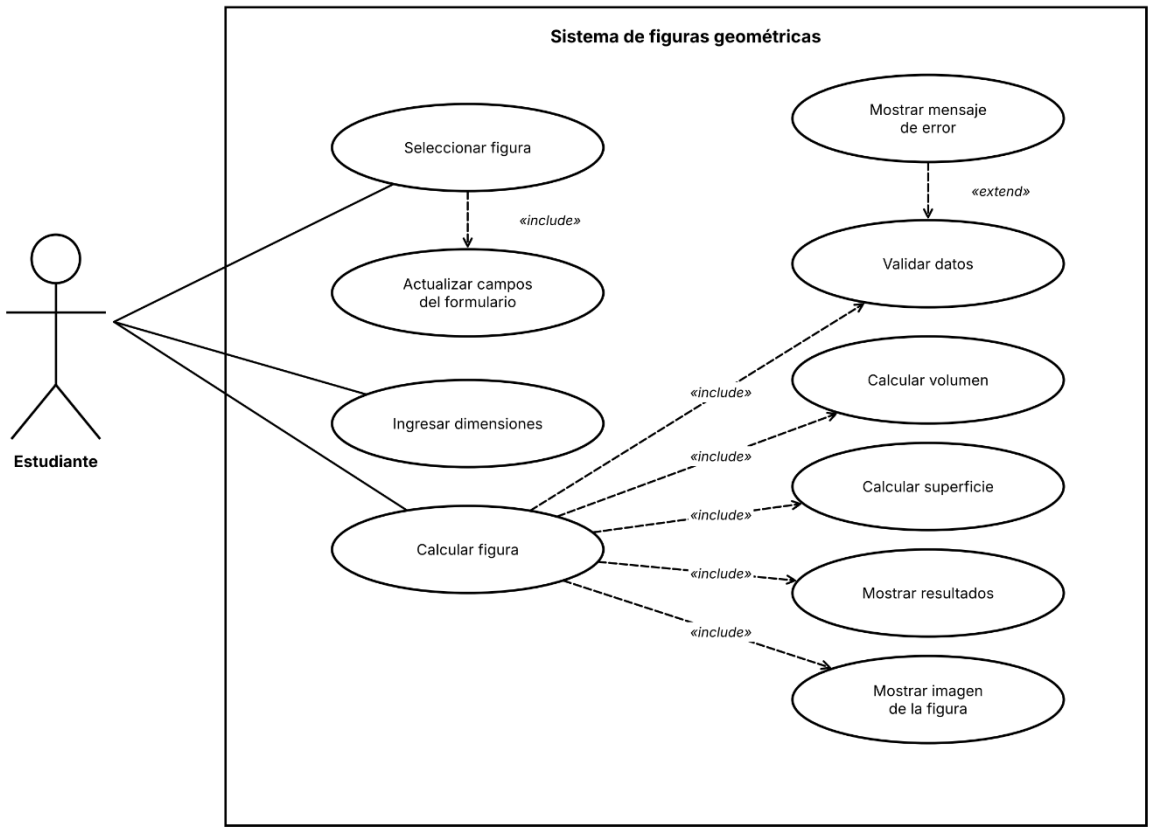


Ejemplo: cálculo de un cilindro con radio = 3 cm y altura = 5 cm

Diagrama de Clases



Casos de uso



Enlace de Github: https://github.com/mabarreraa-eng/Actividad-III-Poo.git